

Data Analytics

A hands-on portfolio of three end-to-end data analytics and machine learning case studies — **Retail Sales EDA**, **Customer Segmentation**, and **House Price Prediction**. Each project follows the same real-world workflow: load and clean raw data, explore it, engineer features, apply a statistical or machine learning technique, and translate the output into a plain-language business recommendation.

The projects were built as part of an internship program (the working directories embedded in the notebooks point to an Infobyte/OIBSIP folder layout, with each project living in its own DataAnalytics-L* sub-folder), but every notebook is fully self-contained and can be run independently.

Project Summary

#	Project	Technique	Dataset Size	Core Question
1	Retail Sales EDA	Descriptive statistics, time-series & correlation analysis	1,000 transactions	Which products, months, and customer groups drive revenue?
2	Customer Segmentation	RFM analysis + K-Means clustering	3,660 transactions → ~1,450 unique customers	Which customers deserve which marketing treatment?
3	House Price Prediction	Linear / Ridge / Lasso Regression	2,000 property records	Can structural & location features predict sale price?

Repository Contents

File	Type	Description
DataAnalytics-L1-EDARetailSales.ipynb	Notebook	Exploratory data analysis of retail sales transactions
retail_sales_dataset.csv	Data	Raw retail transactions (input to the notebook above)
		Customer segmentation

DataAnalytics-L1-ecommerce-dataset.ipynb	Notebook	using RFM analysis + K-Means clustering
ecommerce_dataset_updated.csv	Data	Raw e-commerce transactions (input to the notebook above)
customer_segment_marketing_insights.csv	Data	Derived output: per-segment RFM averages + recommended marketing action
House_Price_Prediction.ipynb	Notebook	Regression modelling to predict house prices
house_prices.csv	Data	Raw housing records (input to the notebook above)
pyvenv.cfg	Config	Metadata for the local Python 3.14.5 virtual environment used during development
_gitignore	Config	Should be renamed to .gitignore; currently ignores everything (*)
LICENSE	Legal	Apache License 2.0

Path note: each notebook reads its dataset from a relative dataset/ sub-folder (e.g. `pd.read_csv("dataset/ecommerce_dataset_updated.csv")`). Create a dataset/ folder next to each notebook and place the matching CSV inside it, or edit the path in the first code cell, before running.

1. Retail Sales Exploratory Data Analysis

Notebook: DataAnalytics-L1-EDARetailSales.ipynb • **Dataset:** retail_sales_dataset.csv (1,000 rows × 9 columns)

Goal: Turn raw point-of-sale transactions into a clear picture of *when* customers buy, *what* they buy, and *who* is buying it — the kind of baseline reporting a retail analytics team would build before deeper modelling.

Columns: Transaction ID, Date, Customer ID, Gender, Age, Product Category, Quantity, Price per Unit, Total Amount.

Workflow: 1. **Data quality check** — shape, dtypes, and a null-value scan (dataset was clean, no missing values). 2. **Descriptive statistics** — mean, median, mode, and standard deviation across all numeric columns. 3. **Time-series analysis** — transactions aggregated by month and by quarter to reveal seasonality. 4. **Customer demographics** — ages bucketed into six bands (0-20, 21-30, 31-40, 41-50, 51-60, 60+) and cross-tabulated with gender. 5. **Product performance** — total revenue ranked by product category. 6. **Correlation analysis** — a Pearson correlation heatmap across Age, Quantity, Price per Unit, and Total Amount. 7. **Business insight** — average spend calculated per age group to flag the highest-value demographic.

Key findings (from the actual notebook output):

Metric	Value
Average transaction value	₹456.00
Average customer age	41.4 years
Average items per transaction	2.51
Gender split	510 Female / 490 Male — near-even
Highest-revenue category	Electronics (₹156,905)
Lowest-revenue category	Beauty (₹143,515) — the three categories are within ~9% of each other
Strongest correlation	Price per Unit ↔ Total Amount ($r = 0.85$) — price, not quantity, drives spend
Best quarter	Q4 2023 (₹126,190), narrowly ahead of Q2 (₹123,735)
Highest-spending age group	0–20 years old (₹551 average spend per transaction)

Key takeaway: Revenue is fairly evenly split across the three product categories — no single category dominates — while spend per transaction is driven mainly by price point rather than basket size. Sales also show a clear quarterly rhythm (Q4 and Q2 outperform Q1 and Q3), which is directly useful for inventory and promotional planning. Interestingly, the youngest age band spends the most per transaction on average, a signal worth validating with more data before acting on it.

2. Customer Segmentation (RFM + K-Means)

Notebook: DataAnalytics-L1-ecommerce-dataset.ipynb · **Dataset:** ecommerce_dataset_updated.csv (3,660 rows × 8 columns)

Goal: Move beyond one-size-fits-all marketing by grouping customers into behavioral segments and pairing each segment with a specific, actionable campaign.

Columns: User_ID, Product_ID, Category, Price (Rs.), Discount (%), Final_Price(Rs.), Payment_Method, Purchase_Date.

Workflow: 1. **Cleaning** — parsed Purchase_Date to datetime, coerced price columns to numeric, and dropped invalid or duplicate rows. 2. **RFM feature engineering** — for every customer, computed: - **Recency** — days since their most recent purchase (relative to one day after the latest date in the dataset). - **Frequency** — total number of

purchases made. - **Monetary** — average amount spent per purchase. 3. **Standardization** — scaled the three RFM features with StandardScaler so that Monetary (naturally on a larger numeric scale) doesn't dominate the clustering. 4. **Clustering** — ran **K-Means** across $K = 1-10$ and used the **Elbow Method** to settle on **$K = 4$** clusters. 5. **Visualization** — scatter plots of Recency vs. Frequency and Frequency vs. Monetary, colored by cluster assignment. 6. **Profiling & labeling** — averaged RFM values per cluster and translated the numbers into human-readable customer archetypes. 7. **Marketing recommendations** — paired every segment with a concrete recommended action, exported to customer_segment_marketing_insights.csv.

Resulting segments (from customer_segment_marketing_insights.csv):

Cluster Size	Customer Type	Avg. Recency (days)	Avg. Frequency	Avg. Monetary (₹)	Recommendation
368	Champions	85.3	1.0	317.20	VIP reward, exclusive discount, product recommendation, referral incentive
404 / 371	(Loyal / Regular-type segments)	—	—	—	See CSV for remaining segment detail
311	At-Risk High-Value Customers	270.1	1.0	321.51	Personalized back can limited-time offers, recommendation based on purchase history

Key takeaway: Four behaviorally distinct groups emerged, each roughly 300–400 customers in size. Notably, **At-Risk customers actually spend slightly more per purchase (₹321.51) than Champions (₹317.20)** — the difference between the two groups is entirely about *recency* (85 vs. 270 days), not spending power. This is a valuable insight: the highest-value re-engagement opportunity in this dataset isn't a low-spender, it's a high-value customer who has simply gone quiet — exactly the kind of customer a win-back campaign should prioritize first.

3. House Price Prediction

Notebook: House_Price_Prediction.ipynb · **Dataset:** house_prices.csv (2,000 rows × 10 columns)

Goal: Build a regression pipeline that predicts a house's sale price from its structural attributes and location, and honestly evaluate whether those features actually explain price variation.

Columns: Id, Area, Bedrooms, Bathrooms, Floors, YearBuilt, Location, Condition, Garage, Price (target).

Workflow: 1. **EDA** — checked shape, dtypes, and missing values (none found); measured target skewness (≈ -0.06 , i.e. the price distribution is roughly symmetric, not the typical right-skew seen in

real housing markets). 2. **Feature selection** — Area, Bedrooms, Bathrooms, Floors, YearBuilt, Location, Condition, and Garage selected as candidate predictors of Price. 3. **Preprocessing pipeline** — built with ColumnTransformer + Pipeline: - Numerical features → median imputation. - Categorical features (Location, Condition, Garage) → most-frequent imputation, then **One-Hot Encoding**. 4. **Correlation analysis** — a heatmap of every encoded feature against Price to gauge linear predictive strength up front. 5. **Train/test split** — 80/20 split (1,600 train / 400 test rows), `random_state=42` for reproducibility. 6. **Modeling** — trained a **Linear Regression** baseline, then compared it against **Ridge** and **Lasso** regularized regression. 7. **Evaluation** — MSE, RMSE, and R^2 score; actual-vs-predicted scatter plot; residual plot; and a coefficient-level feature-impact breakdown.

Results:

Metric	Value
Mean residual (Actual – Predicted)	₹8,623
Residual range	₹-522,579 to ₹507,380
Best model	Ridge Regression
Best R^2 score	≈ -0.0067

Key takeaway: An R^2 this close to zero — and slightly negative — means the model performs essentially the same as (or marginally worse than) simply predicting the average price for every house. This is a genuinely useful *negative result*, not a failure: it indicates that Area, Bedrooms, Bathrooms, Floors, YearBuilt, Location, Condition, and Garage carry little to no linear relationship with Price in this particular dataset — consistent with the data being synthetically or randomly generated rather than sampled from a real housing market. The correct response isn't to force a better linear fit; it's to (a) verify the data-generating process, and (b) if the data is real, test non-linear models capable of capturing interaction effects that a straight-line model cannot.

Tech Stack

Category	Tools
Language	Python 3.14.5
Data handling	pandas
Visualization	matplotlib, seaborn
Machine learning	scikit-learn — StandardScaler, KMeans, LinearRegression, Ridge, Lasso, ColumnTransformer, Pipeline, OneHotEncoder, SimpleImputer, <code>train_test_split</code>
Environment	Local venv (see <code>pyenv.cfg</code>)
Notebook runtime	Jupyter Notebook

Getting Started

1. **Clone or download** this repository.
2. **Create and activate a virtual environment:**

```
python -m venv venv
```

```
# Windows
venv\Scripts\activate

# macOS/Linux
source venv/bin/activate
```

3. Install dependencies:

```
pip install pandas numpy matplotlib seaborn scikit-learn jupyter
```

4. **Arrange the datasets.** For each project, create a dataset/ folder next to its notebook and place the matching CSV inside (or edit the `pd.read_csv(...)` path directly). See [Suggested Folder Structure](#) below.

5. Launch Jupyter and run top to bottom:

```
jupyter notebook
```

Suggested Folder Structure

To match what each notebook expects, organize the repository like this:

```
data-analytics-portfolio/
├── retail-sales-eda/
│   ├── DataAnalytics-L1-EDARetailSales.ipynb
│   └── dataset/
│       └── retail_sales_dataset.csv
├── customer-segmentation/
│   ├── DataAnalytics-L1-ecommerce-dataset.ipynb
│   └── dataset/
│       └── ecommerce_dataset_updated.csv
├── house-price-prediction/
│   ├── House_Price_Prediction.ipynb
│   └── dataset/
│       └── house_prices.csv
├── .gitignore
├── LICENSE
└── README.md
```

Repository Housekeeping Notes

- **Rename `_gitignore` → `.gitignore`.** Git only recognizes the dotfile name. As currently written, the file ignores *everything* (*), which would silently exclude the notebooks and CSVs from version control. If you want them tracked, add explicit negation rules:

```
*
!*.ipynb
!*.csv
!*.md
!.gitignore
```

- **Environment metadata (`pyvenv.cfg`)** points to a Windows-based Python 3.14.5 install — this file doesn't need to be committed to the repo; it's regenerated automatically whenever `venv` is created.
 - **Licensing:** the project is distributed under the **Apache License 2.0** (see `LICENSE`), which permits reuse, modification, and distribution with attribution and a copy of the license notice.
-

License

This project is licensed under the **Apache License 2.0** — see [LICENSE](#) for the full text.